Karlsruher Institut für Technologie

Institut für Informationssicherheit und Verlässlichkeit
Prof. Dr. André Platzer platzer@kit.edu
Prof. Dr. Ralf H. Reussner reussner@kit.edu
Prof. Dr. Ina Schaefer ina.schaefer@kit.edu

Klausur Programmierparadigmen — Beispiellösung

SS25, 26. September 2025, 11:00 – 13:00 Uhr

Bearbeitungszeit: 120 min

Aufgabe 1 (Allgemeine Fragen) [6 Punkte]

Kreuzen Sie die richtigen Antworten an. Es können beliebig viele Antwortmöglichkeiten richtig sein (mindestens jedoch 1). Jede vollständig richtige Antwort gibt einen (1) Punkt. Wenn Sie Ihre Antwort korrigieren möchten, malen Sie das Quadrat aus und setzen Sie das Kreuz neu oder malen Sie es neben ein ausgemaltes Feld.

Parallelprogrammierung

(a) Bei welchen kollektiven MPI-Operationen liegt das Ergebnis nur auf dem Root-Knoten vor?

- ☐ Broadcast
- ☒ Gather
- ☐ Scatter
- ☒ Reduce

(b) Welche der folgenden Operationen sind in Java atomar, d.h. können nicht durch andere Threads unterbrochen werden?

- ☐ `i++` für eine `int`-Variable `i`
- ☒ Lesezugriff auf eine `int`-Variable
- ☐ Schreibzugriff auf eine `double`-Variable
- ☒ Lesezugriff auf eine `volatile long`-Variable

(c) Welche der folgenden Implikationen müssen nach dem Liskovschen Substitutionsprinzip für Methodenveträge gelten?

- ☐ $Precondition_{Subclass} \Rightarrow Precondition_{Superclass}$
- ☒ $Precondition_{Superclass} \Rightarrow Precondition_{Subclass}$
- ☒ $Postcondition_{Subclass} \Rightarrow Postcondition_{Superclass}$
- ☐ $Postcondition_{Superclass} \Rightarrow Postcondition_{Subclass}$

Compiler

(d) Was enthält die FIRST-Menge eines nicht-Terminals in einer Grammatik?

- ☒ Alle Terminale, die am Anfang einer Ableitung dieses nicht-Terminals auftreten können.
- ☐ Alle nicht-Terminale, von denen dieses nicht-Terminal abgeleitet werden kann.
- ☐ Alle Terminale und nicht-Terminale, die in der Ableitung auftreten können.
- ☐ Die erste Ableitungsregel, die dieses nicht-Terminal enthält.

(e) Wofür steht das *LL* in LL(k)-Grammatik?

- ☒ Left-to-right scanning, Leftmost derivation.
- ☐ Leftmost derivation, Lookahead parsing.
- ☐ Lexical analysis, Left-to-right scanning.
- ☐ Language lexing, Lookahead derivation.

(f) Was sind die Vorteile von Symboltabellen?

- ☒ Symboltabellen speichern Informationen über Variablen, Funktionen und Typen. Damit machen sie die Kompilierung effizienter.
- ☐ Symboltabellen helfen bei der Optimierung von Laufzeitperformance, indem sie die tatsächlichen Speicherwerte von Variablen speichern.
- ☒ Symboltabellen unterstützen bei der Fehlerüberprüfung, indem sie Typkorrektheit und Scoping-Regeln überprüfen.
- ☐ Symboltabellen werden primär für Debugging und das Nachverfolgen von Variablenwerten zur Laufzeit verwendet.

```
-- Erinnerung: Typsignaturen
map :: (a -> b) -> [a] -> [b]
filter :: (a -> Bool) -> [a] -> [a]
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
foldl :: Foldable t => (b -> a -> b) -> b -> t a -> b
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
all :: Foldable t => (a -> Bool) -> t a -> Bool
any :: Foldable t => (a -> Bool) -> t a -> Bool
```

Aufgabe 2 (Haskell: Matrix) [17 Punkte]

Matrizen sind zweidimensionale Arrays und ein Schlüsselkonzept der Linearen Algebra. In Haskell können Matritzen als Liste von Zeilenvektoren dargestellt werden. So lässt sich z.B. $M = \left(\begin{smallmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{smallmatrix}\right)$ darstellen als

```
type Matrix t = [[t]]
M = [[1, 2, 3],
     [4, 5, 6]]
```

Für diese Aufgabe können Sie annehmen, dass alle Matrizen wohlgeformt sind. Das bedeutet: Jede Matrix enthält keine leere Zeile, und alle Zeilen einer Matrix haben die gleiche Anzahl an Elementen.

(a) Die Größe einer Matrix ist gegeben durch die Anzahl an Zeilen und Spalten der Matrix. So ist z.B. M eine 2×3 Matrix. [2 Punkte]
Implementieren Sie die Funktion

```
size :: Matrix t -> (Int, Int)
```

die die Größe einer Matrix zurückgibt.

Beispiel:
`size M = (2,3)`

Beispiellösung:

```
size :: Matrix t -> (Int, Int)
size [] = (0,0)
size m  = (length m, length (head m))
```

(b) Implementieren Sie die Funktion [3 Punkte]

```
extractFstCol :: Matrix t -> ([t], Matrix t)
```

die sowohl die erste *Spalte* einer Matrix als auch die ursprüngliche Matrix ohne erste Spalte zurückgibt.

Achten Sie darauf, dass die Restmatrix wohlgeformt ist, wenn die ursprüngliche Matrix nur eine Spalte hat.

Beispiel:

```
extractFstCol M = ([1,4], [[2,3],
                           [5,6]])
```

Beispiellösung:

```
extractFstCol :: Matrix t -> ([t], Matrix t)
extractFstCol [] = error "No␣column␣to␣extract"
extractFstCol m  = (map head m, filter (not . null) (map tail m))

extractFstCol2 [] = error "No␣column␣to␣extract"
extractFstCol2 m
  | snd (size m) == 1 = (map head m, [])
  | otherwise         = (map head m, map tail m)
```

(c) Implementieren Sie die Funktion [3 Punkte]

```
transpose :: Matrix t -> Matrix t
```

die die Transponierte der Matrix zurückgibt, d.h. die Matrix bei der Zeilen und Spalten vertauscht wurden.

Beispiel:

```
transpose M = [[1, 4],
               [2, 5],
               [3, 6]]
```

Beispiellösung:

```
transpose :: Matrix t -> Matrix t
transpose []      = []
transpose ([]:xs) = [] -- not needed
transpose m = col : (transpose rest)
  where (col, rest) = extractFstCol m
```

(d) Für einen Vektor $v = (v_i)_{1 \leq i \leq m}$ der Größe m und eine $m \times n$ Matrix $A = (a_{ij})_{1 \leq i \leq m, 1 \leq j \leq n}$ ist das *linksseitge* Produkt $v^\intercal M$ der Vektor $(w_j)_{1 \leq i \leq n}$ mit Größe $\boldsymbol{n}$ wobei $w_j = \sum_{1 \leq i \leq m} v_i \cdot a_{ij}$. [5 Punkte]

Implementieren Sie die Funktion

```
lmul :: Num t => [t] -> Matrix t -> [t]
```

die das linksseitge Produkt eines Vektors mit einer Matrix berechnet.

Hinweis:
Die Berechnung von w_j hängt nur von v und der j-ten Spalte von A ab.

Beispiel:
`lmul [2,-1] M = [-2, -1, 0]`

Beispiellösung:

```
lmul :: Num t => [t] -> Matrix t -> [t]
lmul v m
  | length v == fst (size m) = map (sum . (zipWith (*) v)) (transpose m)
  | otherwise                = error "Size␣do␣not␣match"
```

(e) Für eine $m \times n$ Matrix $A = (a_{ij})_{1 \leq i \leq m, 1 \leq j \leq n}$ und eine $n \times p$ Matrix $B = (b_{jk})_{1 \leq j \leq n, 1 \leq k \leq p}$ ist das Matrixprodukt die $\boldsymbol{m} \times \boldsymbol{p}$ Matrix $C = (c_{ik})_{1 \leq i \leq m, 1 \leq k \leq p}$ wobei $c_{ik} = \sum_{1 \leq j \leq n} a_{ij} \cdot b_{jk}$. [4 Punkte]

Implementieren Sie die Funktion

```
mul :: Num t => Matrix t -> Matrix t -> Matrix t
```

die das Produkt zweier Matritzen berechnet.

Hinweis:
Die i-te Zeile der ersten Matrix A wird nur für die i-te Zeile des Ergebnises benötigt.

Beispiel:

```
mul [[2,-1], [-1,0]] M = [[-2, -1, 0],
                          [-1, -2, -3]]
```

Beispiellösung:

```
mul :: Num t => Matrix t -> Matrix t -> Matrix t
mul m n
  | snd (size m) == fst (size n) = map (`lmul` n) m
  | otherwise                    = error "Size␣do␣not␣match"
```

Aufgabe 3 (Haskell: Baumtraversierung) [13 Punkte]

Betrachten Sie den folgenden algebraischen Datentyp für Binärbäume:

```
data Tree t = L | N (Tree t) t (Tree t)
```

Für diese Aufgabe können Sie annehmen, dass alle Knotenschlüssel unterschiedlich sind.

(a) Implementieren Sie die Funktion [4 Punkte]

```
depth :: Eq a => Tree a -> a -> Maybe Int
```

die für ein Element seine Tiefe im Baum berechnet oder `Nothing` zurückgibt, falls das Element nicht gefunden wurde. Der Wurzelknoten hat Tiefe 0. Sie können für diese Aufgabe endliche Bäume annehmen.

Beispiel:

```
depth (N L 0 (N (N L 2 L) 1
                (N L 3 (N L 4 L)))) 3 = Just 2
```

Beispiellösung:

```
depth :: Eq a => Tree a -> a -> Maybe Int
depth L _ = Nothing
depth (N l s' r) s
  | s' == s   = Just 0
  | otherwise = go (depth l s) (depth r s)
    where go (Just d) _ = Just (d+1)
          go _ (Just d) = Just (d+1)
          go Nothing Nothing = Nothing

depth2 L _ = Nothing
depth2 (N l s' r) s
  | s' == s   = Just 0
  | otherwise = case (depth2 r s) of
      Just x  -> Just (x+1)
      Nothing -> case (depth2 l s) of
        Just x  -> Just (x+1)
        Nothing -> Nothing
```

(b) Wir erweitern `depth` nun für unendliche Binärbäume: [9 Punkte]

Implementieren Sie die Funktion

```
depthInf :: Eq a => Tree a -> a -> Maybe Int
```

die wie `depth` die Tiefe eines Elements im Baum berechnet jedoch auch für unendliche Binärbäume genutzt werden kann: Die Funktion terminiert immer mit dem korrekten Ergebnis falls das Element im Baum enthalten ist. Die Funktion muss nicht terminieren, wenn das gesuchte Element nicht im Baum enthalten ist.

Hinweis:
Führen Sie eine Breitensuche aus. Nutzen Sie eine Liste um die Knoten zu speichern, die als nächstes besucht werden. Achten Sie darauf, dass die Knoten in der richtigen Reihenfolge entnommen werden.

Beispiel:

```
    treeInf n = N L n (treeInf (n + 1))
    depthInf (N (treeInf 2) 1 (N L 0 L)) 0 = Just 1
```

Beispiellösung:

```
depthInf :: Eq a => Tree a -> a -> Maybe Int
depthInf tree x = bfs [(0,tree)]
  where
    bfs [] = Nothing
    bfs ((_,L):xs) = bfs xs
    bfs ((dep, N l y r):xs)
      | x == y    = Just dep
      | otherwise = bfs (xs ++ [(dep+1,l), (dep+1,r)])
```

Aufgabe 4 (Lambda) [24 Punkte]

Erinnerung: Die folgenden Regeln definieren wie Tupel vom Summentyp eingeführt werden können ($+\mathrm{I}_1$,$+\mathrm{I}_2$) und Terme vom Summentyp eliminiert werden können ($+\mathrm{E}$), (wobei u, w Variables, M, N, O Terme, und A, B, C Typen sind):

$$(+\mathrm{I}_1)\ \frac{M : A}{\mathbf{inl}\, M : A + B} \qquad (+\mathrm{I}_2)\ \frac{M : B}{\mathbf{inr}\, M : A + B} \qquad (+\mathrm{E})\ \frac{M : A + B \quad \begin{array}{c}\overline{u : A}\\ \vdots \\ N : C\end{array} \quad \begin{array}{c}\overline{w : B}\\ \vdots \\ O : C\end{array}}{\mathbf{case}\, M\, \mathbf{of}\, \mathbf{inl}\, u \Rightarrow N \mid \mathbf{inr}\, w \Rightarrow O : C}$$

(a) Vervollständigen Sie folgende Reduktion, die das Resultat dessen vereinfacht, wie die $+\mathrm{E}$ Elimination direkt auf das Resultat der $+\mathrm{I}_1$ Introduktion angewandt wird (wobei $\mathcal{D}$, $\mathcal{E}$, $\mathcal{F}$ beliebige Ableitungen sind): [5 Punkte]

$$+\mathrm{E}\,\frac{+\mathrm{I}_1\,\frac{\begin{array}{c}\mathcal{D}\\ M : A\end{array}}{\mathbf{inl}\, M : A + B} \quad \begin{array}{c}\overline{u : A}\\ \mathcal{E}\\ N : C\end{array} \quad \begin{array}{c}\overline{w : B}\\ \mathcal{F}\\ O : C\end{array}}{\mathbf{case}\, \mathbf{inl}\, M\, \mathbf{of}\, \mathbf{inl}\, u \Rightarrow N \mid \mathbf{inr}\, w \Rightarrow O : C} \quad \rightsquigarrow \quad \ldots$$

Beispiellösung:

$$\begin{array}{c}\begin{array}{c}\mathcal{D}\\ \overline{u : A}\end{array}\\ \mathcal{E}[u \mapsto M]\\ N[u \mapsto M] : C\end{array}$$

(b) Welches Prinzip von Typsystemen begründet obige Reduktion? [2 Punkte]

Beispiellösung:

Korrekte Fallunterscheidungen von in Alternativen eingebettete Typen werden durch die obige Beweisreduktion begründet also, dass Fallunterscheidungen für bekannte Fälle kollabieren zum konstruierten Fall (nach links und rechts von in-left Einbettungen den linken Fall ergeben, nach entsprechender Substitution).

Erinnerung: Die folgenden Regeln definieren wie λ-Abstraktionen vom Funktionstyp eingeführt werden können ($\to$I), und Terme vom Funktionstyp auf andere Terme angewandt werden können ($\to$E), (wobei u eine Variable, M, N Terme, und A, B Typen sind):

$$(\to\mathrm{I})\ \frac{\begin{array}{c}\overline{u : A}\\ \vdots \\ M : B\end{array}}{\lambda u.M : A \to B} \qquad (\to\mathrm{E})\ \frac{M : A \to B \quad N : A}{M\,N : B}$$

(c) Welche Beweisreduktion begründet die β-reduktion von λ-Abstraktionen? Geben Sie die passende Beweisreduktion an und begründen Sie den Zusammenhang. [7 Punkte]

Beispiellösung:

eine passende Reduktion an, die das Resultat dessen vereinfacht, wie die $\to$E Elimination direkt auf das Resultat der $\to$I Introduktion angewandt wird:

$$\to\mathrm{E}\,\frac{\to\mathrm{I}\,\dfrac{\begin{array}{c}\overline{u : A}\\ \mathcal{D}\\ M : B\end{array}}{\lambda u.M : A \to B} \quad \begin{array}{c}\mathcal{E}\\ N : A\end{array}}{(\lambda u.M)\,N : B} \quad \rightsquigarrow \quad \begin{array}{c}\mathcal{E}\\ \overline{u : A}\\ \mathcal{D}[u \mapsto N]\\ M[u \mapsto N] : B\end{array}$$

Diese logische Reduktion begründet genau die logische Korrektheit der β-Reduktion im λ-Kalkül der Terme.

Erinnerung: Beispielnotation für Konstruktoren und Destruktoren der Programmtypen.

Name	**Konstruktor**	: **Typ**	**Destruktor**
Produkt	$\langle M, N \rangle$	$: A \times B$	$\mathbf{fst}\, M, \mathbf{snd}\, M$
Funktion	$\lambda u.M$	$: A \to B$	$M\, N$
disj. Summe	$\mathbf{inl}\, M, \mathbf{inr}\, M$	$: A + B$	$\mathbf{case}\, M\, \mathbf{of}\, \mathbf{inl}\, u \Rightarrow N \mid \mathbf{inr}\, w \Rightarrow O$
Unit	$\langle\rangle$	$: \mathbf{1}$	
Empty		$: \mathbf{0}$	$\mathbf{abort}\, M$

Für jeden der folgenden Typen geben Sie ein (terminierendes) funktionales Programm als λ-Term diesen Typs an oder begründen Sie *kurz* warum es solch ein Programm nicht gibt.

(d) $A + B \to B + A$ [2 Punkte]

Beispiellösung:

$\lambda e.\mathbf{case}\, e\, \mathbf{of}\, \mathbf{inl}\, a \Rightarrow \mathbf{inr}\, a \mid \mathbf{inr}\, b \Rightarrow \mathbf{inl}\, b$

(e) $A \times B \to B + A$ [2 Punkte]

Beispiellösung:

$\lambda u.\mathbf{inr}\, \mathbf{fst}\, u$ or $\lambda u.\mathbf{inl}\, \mathbf{snd}\, u$

(f) $(A \to B) \to (B \to A)$ [2 Punkte]

Beispiellösung:

Das geht nicht weil nicht alle Funktionen umdrehbar sind, wenn etwa $A = \mathbf{0}$ unmöglich aber $B = \mathbf{1}$ möglich ist kann das unmögliche A nicht aus dem möglichen B konstruiert werden.

(g) $A \to ((A \to B) \to B)$ [2 Punkte]

Beispiellösung:

$\lambda a.\lambda f.f\, a$

(h) $(A \to B) \to (A \times B)$ [2 Punkte]

Beispiellösung:

Das ist unmöglich wenn $A = \mathbf{0}$ unmöglich ist kann man kein A aus dem Nichts herzaubern.

Aufgabe 5 (MPI) [9 Punkte]

(a) Betrachten Sie den Programm-Code auf der nächsten Seite. Füllen Sie die untenstehenden Tabellen mit den Werten aus, die die angegebenen Variablen nach Ausführung der angegebenen Zeile haben. Schreiben Sie Leerzeichen in Strings als "␣". Zu Beginn der Ausführung liegen die Daten nur auf dem root-Prozess vor, wie in der ersten Tabelle angegeben ("-" bezeichnet hier uninitialisierten Speicher). [3 Punkte]

In dem Programm-Code werden Worte als Strings aus ASCII-Zeichen fester Länge repräsentiert. Falls ein Wort kürzer als die vorgegebene String-Länge ist, wird rechts mit Leerzeichen aufgefüllt. Die Anzahl an Vokalen (engl. *vowels*) wird als Integer-Array der Länge 5 dargestellt, wobei die Zahlen darin die Anzahl der Buchstaben *a*, *e*, *i*, *o* und *u* beschreiben. *Beispiel:* Für das Wort "propa␣␣␣" gibt die Funktion `countVowels` das Array { 1, 0, 0, 1, 0 } zurück.

Variable: `words`

Prozess Nr.	Wert nach Zeile 16
0	"hello␣␣␣world␣␣␣abc␣␣␣␣␣you␣␣␣␣␣"
1-3	-

Variable: `localData`

Prozess Nr.	Wert nach Zeile 26 (Lösung)
0	"hello␣␣␣"
1	"world␣␣␣"
2	"abc␣␣␣␣␣"
3	"you␣␣␣␣␣"

Variable: `localVowels`

Prozess Nr.	Wert nach Zeile 34 (Lösung)
0	{ 0, 1, 0, 1, 0 }
1	{ 0, 0, 0, 1, 0 }
2	{ 1, 0, 0, 0, 0 }
3	{ 0, 0, 0, 1, 1 }

Variable: `globalVowels`

Prozess Nr.	Wert nach Zeile 40 (Lösung)
0	{ 1, 1, 0, 3, 1 }
1	{ 0, 0, 0, 0, 0 }
2	{ 0, 0, 0, 0, 0 }
3	{ 0, 0, 0, 0, 0 }

Beispiellösung: Siehe Tabellen.

```
1  const int ROOT = 0;
2  const int STRING_LENGTH = 8;
3  const int NUMBER_OF_VOWELS = 5;
4
5  // allocate memory for an array of num objects of size
6  // and initialize all bytes in the allocated storage to zero
7  void* calloc( size_t num, size_t size );
8
9  // count the vowels in word
10 // returns an array of length 5 with the number of As at index 0,
11 //     number of Es at index 1, ...
12 // example: countVowels("propa␣␣␣") returns { 1, 0, 0, 1, 0 }
13 int* countVowels(char *word);
14
15 // count the vowels in count words
16 int* countVowelsParallel(char *words, int count) {
17     int size, rank;
18     MPI_Comm_size(MPI_COMM_WORLD, &size);
19     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
20
21     int localLength = count * STRING_LENGTH / size;
22     char *localData = calloc(localLength, sizeof(char));
23     MPI_Scatter(
24         words, localLength, MPI_CHAR,
25         localData, localLength, MPI_CHAR,
26         ROOT, MPI_COMM_WORLD);
27
28     int *localVowels = calloc(NUMBER_OF_VOWELS, sizeof(int));
29     for (int i = 0; i < localLength; i += STRING_LENGTH) {
30         int *wordVowels = countVowels(localData + i, STRING_LENGTH);
31         for (int j = 0; j < NUMBER_OF_VOWELS; j++) {
32             localVowels[j] += wordVowels[j];
33         }
34     }
35
36     int *globalVowels = calloc(NUMBER_OF_VOWELS, sizeof(int));
37     MPI_Reduce(
38         localVowels, globalVowels,
39         VOWELS, MPI_INT, MPI_SUM,
40         ROOT, MPI_COMM_WORLD);
41
42     return globalVowels;
43 }
```

(b) Implementieren Sie eine Reduce-Funktion, die zu dem gleichen Ergebnis wie `MPI_Reduce` mit den Argumenten `MPI_INT` und `MPI_SUM` führt. Verwenden Sie dafür ausschließlich die MPI-Funktionen `MPI_Send` und `MPI_Recv`. Die Signaturen der beiden Funktionen finden Sie in der Lösungsvorlage auf der nächsten Seite. [6 Punkte]

Beispiellösung:

```c
void MY_Reduce_Sum(
        int *sendbuf, int *recvbuf, int count,
        int root, MPI_Comm comm) {
    int size, rank;
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    int tag = 42;

    // send
    if (rank == root) {
        // copy directly if we're on root
        for (int i = 0; i < count; i++) {
            recvbuf[i] = sendbuf[i];
        }
    } else {
        // else send all data to root
        MPI_Send(sendbuf, count, MPI_INT, root, tag, comm);
    }

    // recv
    if (rank == root) {
        // temporary receive buffer
        int *buf = calloc(count, sizeof(int));
        // for each other node
        for (int i = 0; i < size; i++) {
            // not root, as we did not send any data to root
            if (i == root)
                continue;

            // receive all data from other node i
            MPI_Recv(
                buf, count, MPI_INT,
                i, tag, comm, MPI_STATUS_IGNORE);
            // add each number from the temporary receive buffer
            // to actual receive buffer
            for (int j = 0; j < count; j++) {
                recvbuf[j] += buf[j];
            }
        }
    }
}
```

Aufgabe 6 (Java Parallelprogrammierung) [10 Punkte]

In dieser Aufgabe betrachten wir Regeln (`Rule`), die auf einen Datenbestand angewendet werden, um einzelne Datenpunkte (`Data`) zu bearbeiten. Regeln sind nur auf bestimmte Datenpunkte anwendbar (`boolean Rule::isApplicable(Data)`) und können Abhängigkeiten zu anderen Regeln haben (`Stream<Rule> Rule::dependsOn()`). Eine Regel kann erst ausgeführt werden, wenn die Regeln, von denen sie abhängt, vollständig ausgeführt sind.

Nehmen Sie an, dass die verwendeten Datenstrukturen nicht synchronisiert sind, also gleichzeitige Zugriffe von mehreren Threads extern synchronisiert werden müssen.

(a) Um die Abarbeitung der Regeln zu beschleunigen, soll deren Ausführung parallelisiert werden. Dazu wird eine Form des Producer-Consumer-Muster verwendet. Ein einzelner Thread (`prepareRules`) überprüft, welche Regeln bereit zur Ausführung sind und legt diese in eine Warteschlange. Aus dieser nehmen sich mehrere Threads (`executeRules`) ausführbare Regeln und führen diese auf passenden Datenpunkten aus. Es darf immer nur ein Thread gleichzeitig auf einen Datenpunkt zugreifen. [3 Punkte]

Betrachten Sie die nachfolgende Implementierung und geben Sie die kritischen Abschnitte an, die für die parallele Ausführung der beiden Methoden geschützt werden müssen. Tragen Sie die entsprechenden Zeilennummern, sowie eine geeignete Synchronisationsvariable für ein Monitorobjekt in die Tabelle auf der nächsten Seite ein.

Hinweis: Die Tabelle enthält mehr Zeilen, als für eine korrekte Lösung nötig sind.

```
1  private Set<Data> dataSet;
2  private Set<Rule> openRules;
3  private Queue<Rule> readyRules;
4
5  public void prepareRules() {
6      while (!openRules.isEmpty()) {
7          for (Rule rule : openRules) {
8              if (rule.dependsOn().noneMatch(openRules::contains)) {
9                  openRules.remove(rule);
10                 readyRules.add(rule);
11             }
12         }
13     }
14 }
15 public void executeRules() {
16     while (true) {
17         Rule rule;
18
19         while (readyRules.isEmpty()) {}
20         rule = readyRules.poll();
21
22         for (Data data : dataSet) {
23             if (rule.isApplicable(data)) {
24                 rule.execute(data);
25             }
26         }
27     }
28 }
```

Beispiellösung: Siehe Tabelle.

Zeilennummern (von – bis)	**Synchronisationsvariable**
10	`readyRules`
19 – 20	`readyRules`
23 – 25	`data`

(b) Betrachten Sie noch einmal den Programm-Code auf der vorherigen Seite. In der bisherigen Implementierung warten die Consumer-Threads (`executeRules`) in Zeile 19 aktiv darauf, dass Elemente in die Warteschlange eingefügt werden. [2 Punkte]

Beschreiben Sie, wie der Programm-Code geändert werden müsste, um die Consumer-Threads während der Wartezeit in einen nicht-ausführbaren Zustand (*not runnable*) zu bringen. Sie müssen dafür keinen Programm-Code schreiben. Geben Sie für die Änderungen genaue Zeilennummern und, falls nötig, die Namen von zusätzlich aufgerufenen Methoden an an. Gegebenenfalls notwendige Synchronisationsvariablen (vgl. Teilaufgabe (a)) müssen Sie nicht angeben.

Beispiellösung:

Die Consumer-Threads rufen innerhalb des Schleifenrumpfs in Zeile 19 die Methode `wait` auf, wobei auftretende `InterruptedException`s gefangen werden. Der Producer-Thread ruft nach Zeile 10 die Methode `notifyAll` auf.

(c) Was versteht man unter einem *Memory Consistency Error*? Wodurch kann in Java vermieden werden, dass es beim Zugriff durch verschiedene Threads auf gemeinsam genutzte Daten zu Memory Consistency Errors kommt? Nennen Sie genau eine Maßnahme. [2 Punkte]

Beispiellösung:

Unter einem Memory Consistency Error versteht man einen Fehler, der dadurch ausgelöst wird, dass Threads unterschiedliche Zustände der gleichen Daten sehen. Dies kann zum Beispiel durch CPU-Caching oder das Umordnen von Programminstruktionen ausgelöst werden.

Memory Consistency Errors können in Java vermieden werden, indem eine *Happens-Before*-Beziehung zwischen den Statements hergestellt wird, die auf die Daten zugreifen. Eine Happens-Before-Beziehung kann zum Beispiel durch die Verwendung von `volatile` für Variablen oder durch den Zugriff auf Daten ausschließlich in `synchronized`-Blöcken sichergestellt werden *(nur eine Maßnahme gefordert)*.

(d) Bevor die Datenpunkte (`Data`) verarbeitet werden können, findet ein Vorverarbeitungsschritt statt. Zu Beginn kann ein Datenpunkt einen von drei Zuständen haben: `Valid`, `Invalid` oder `Unresolved`. Die Ausführung von `resolve` auf einem Datenpunkt im Zustand `Unresolved` gibt mehrere Datenpunkte zurück, die den Zustand `Valid` haben. Das Verhalten von `resolve` auf Datenpunkten im Zustand `Valid` oder `Invalid` ist undefiniert. Der ursprüngliche Datenpunkt ist nach dem Aufruf von `resolve` im Zustand `Invalid`. [3 Punkte]

Vervollständigen Sie den nachfolgenden Stream-Ausdruck, so dass ein Stream mit allen validen Datenpunkten zurückgegeben wird, einschließlich der durch `resolve` aus Datenpunkten im Zustand `Unresolved` entstehenden Datenpunkten.

Beispiellösung:

```
Stream<Data> prepareData(Stream<Data> dataset) {
    return dataset
        .filter(data -> data.getState() != State.Invalid)
        .flatMap(data -> data.getState() == State.Unresolved
            ? data.resolve()
            : Stream.of(data));
}
```

Aufgabe 7 (Design by Contract) [11 Punkte]

Im Folgenden finden Sie die Klassenstruktur für die Verwaltung eines Car-Sharing-Unternehmens. Darin können Nutzer (`User`) über ein Buchungssystem (`BookingSystem`) Buchungen (`Booking`) für Autos (`Car`) anlegen. Autos können entweder verschlossen oder offen sein (`Car::isOpen()`) und haben eine Position (`Position`). Im Folgenden betrachten wir Verträge für die Methoden `open(User)` und `close()` in Subklassen von `Car`, die das Auto öffnen, bzw. verschließen.

```
1  abstract class Car {
2      public final BookingSystem system;
3
4      public boolean isOpen() { /* ... */ }
5      public Position getPosition() { /* ... */ }
6
7      public abstract void open(User user);
8      public abstract void close();
9  }
10 interface User { int getAge(); /* ... */ }
11 interface Booking { User getUser(); /* ... */ }
12 interface Zone { boolean isWithin(Position position); }
13 interface Position { /* ... */ }
14 interface BookingSystem {
15     Booking getBooking(Car car, LocalDateTime dateTime);
16     Zone getFreeFloatingZone();
17     /* ... */
18 }
```

(a) Ein reguläres Auto der Car-Sharing-Flotte kann von einem Nutzer geöffnet werden, wenn für den aktuellen Zeitpunkt (`LocalDateTime.now()`) eine Buchung für den entsprechenden Nutzer vorliegt. Nach dem Öffnen soll das Auto offen sein. [3 Punkte]

Betrachten Sie den nachfolgenden JML-Vertrag für die Methode `open(User)`. Entspricht der Vertrag den Anforderungen aus der Aufgabenstellung? Falls nein, beschreiben Sie die notwendigen Änderungen durch Hinzufügen oder Entfernen von Bedingungen. Geben Sie gegebenenfalls die Zeilennummern der Bedingungen an.

```
1  class RegularCar extends Car {
2      /*@ requires !isOpen();
3        @ requires system.getBooking(this, LocalDateTime.now())
4                          != null
5                 && system.getBooking(this, LocalDateTime.now())
6                          .getUser() == user;
7        @*/
8      @Override
9      public void open(User user) { /* ... */ }
10     /* .... */
11 }
```

Beispiellösung:

```
class RegularCar extends Car {
    /*@ requires system.getBooking(this, LocalDateTime.now())
                        != null
                 && system.getBooking(this, LocalDateTime.now())
                        .getUser() == user;
      @ ensures isOpen();
      @*/
    @Override
    public void open(User user) { /* ... */ }
    /* .... */
}
```

(b) Zusätzlich zu den regulären Autos der Car-Sharing-Flotte gibt es auch sogenannte Free-Floating-Autos. Im Unterschied zu regulären Autos können diese auch geöffnet werden, wenn aktuell keine Buchung für das Auto vorliegt (`system.getBooking(...) == null`). In diesem Fall soll die Methode `open` eine entsprechende Buchung für den aktuellen Zeitpunkt anlegen. [6 Punkte]

Vor dem Verschließen eines Free-Floating-Autos muss sichergestellt werden, dass es sich innerhalb der Free-Floating-Zone (`system.getFreeFloatingZone()`) befindet. Nach dem Verschließen eines Free-Floating-Autos soll dieses geschlossen sein. Außerdem soll die Buchung nach dem Verschließen beendet sein, d.h. keine Buchung für den aktuellen Zeitpunkt mehr vorliegen.

Schreiben Sie präzise JML-Verträge für beide Methoden, die diese Anforderungen beschreiben.

Beispiellösung:

```
class FreeFloatingCar extends Car {
    /*@ requires system.getBooking(this, LocalDateTime.now())
                        == null
                || system.getBooking(this, LocalDateTime.now())
                        .getUser() == user
      @ ensures isOpen();
      @ ensures system.getBooking(this, LocalDateTime.now())
                        != null
               && system.getBooking(this, LocalDateTime.now())
                       .getUser() == user;
      @*/
    @Override
    public void open(User user) { /* ... */}

    /*@ requires system.getFreeFloatingZone()
                        .isWithin(getPosition());
      @ ensures !isOpen();
      @ ensures system.getBooking(this, LocalDateTime.now())
                       == null;
      @*/
    @Override
    public void close() { /* ... */ }
}
```

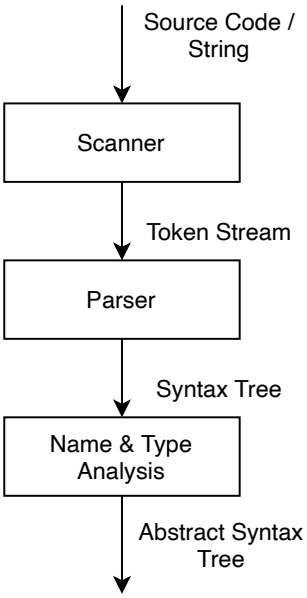
(c) Die Implementierungen der Methode `open` in den beiden Subklassen `RegularCar` und `FreeFloatingCar` von `Car` haben unterschiedliche Vorbedingungen. Angenommen Sie erwarten, dass weitere Subklassen von `Car` hinzukommen werden, wobei Sie keine Annahmen über die Vorbedingungen der von ihnen implementierten Methoden machen können. Welche Vorbedingung müssen Sie für die Methode `open` in der Klasse `Car` wählen, damit das Liskovsche Substitutionsprinzip auch nach dem Hinzufügen neuer Subklassen erfüllt bleibt? Begründen Sie. [2 Punkte]

Beispiellösung: Die Vorbedingungen von überschriebenen Methoden in Subklassen dürfen nur gleich stark oder schwächer als die Vorbedingungen der überschriebenen Methoden sein. Da uns die Vorbedingungen der überschreibenden Methoden nicht bekannt sind, müssen wir `false` als stärkstmögliche Vorbedingung wählen.

Aufgabe 8 (Compiler) [30 Punkte]

(a) Skizzieren Sie den wesentlichen Aufbau des Compiler-Frontends. Zeichnen Sie dabei die verschiedenen Compiler-Bestandteile, Verbindungen und die entsprechenden Ein-/Ausgaben ein. [6 Punkte]

Beispiellösung:



(b) Geben Sie reguläre Ausdrücke an, die die folgende(n) Sprache(n) beschreiben. [8 Punkte]

- Die Menge aller Wörter $w \in \{w, x, y, z\}^*$, bei denen jedem y mindestens zwei z direkt vorangehen und x immer nur in x-Dreiergruppen vorkommt. (3P)

  Beispiellösung:

  [zzy|xxx|w|z]*

- Die Sprache aller automatisch generierten Passwörter, die aus einem oder mehreren Phrasen bestehen können. Bei mehreren Phrasen sind diese mit einem Bindestrich verbunden. Phrasen bestehen dabei aus einem Großbuchstaben gefolgt von mindestens einem Kleinbuchstaben und am Ende ein Digit. (2P)

  Beispiellösung:

  [A-Z][a-z]+[0-9](-[A-Z][a-z]+[0-9])*

- Die Sprache aller Conventional Commit Messages mit den folgenden Regeln. (3P)

  – Jede Commit Message muss mit einem der folgenden Prefixe ihren Typ angeben: `feat`, `fix` oder `revert`.

  – Darauf folgt eine Versionsnummer nach dem Schema [Major].[minor] wobei sowohl Major und minor aus ein oder zwei Digits bestehen können.

  – Darauf folgt eine beliebige Commit Message bestehend aus Buchstaben, Zahlen oder Leerzeichen. (Hinweis: Leerzeichen können als \s enkodiert werden.)

  Beispiellösung:

  (feat | fix | revert)[0-9]?[0-9].[0-9]?[0-9]([a-Z]|[0-9]|\s)*

(c) Gegeben die folgende Grammatik: [1 Punkt]

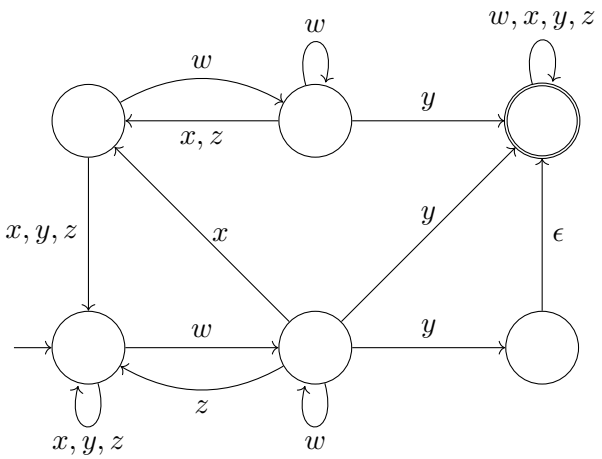
$$S \rightarrow aSbb|\varepsilon$$

Welche Sprache beschreibt diese Grammatik?

Beispiellösung:

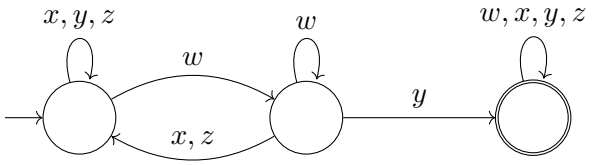
Die Sprache aller Worte mit a's gefolgt von doppelt so vielen b's über das Alphabet $\{a, b\}$.

(d) Welche Sprache beschreibt der folgende nicht-reduzierte Automat? Zeichnen Sie einen minimalen Automaten, der die gleiche Sprache akzeptiert. Hinweis: ϵ ist das leere Wort. [3 Punkte]



Beispiellösung:

Äquivalenter reduzierter Automat:



Damit ist das die Sprache aller Wörter über das Alphabet $\{w, x, y, z\}$, die mindestens ein xy enthalten.

(e) Nennen Sie zwei aus der Vorlesung bekannte Methoden für Garbage Collection und erklären Sie diese kurz. [3 Punkte]

Beispiellösung:

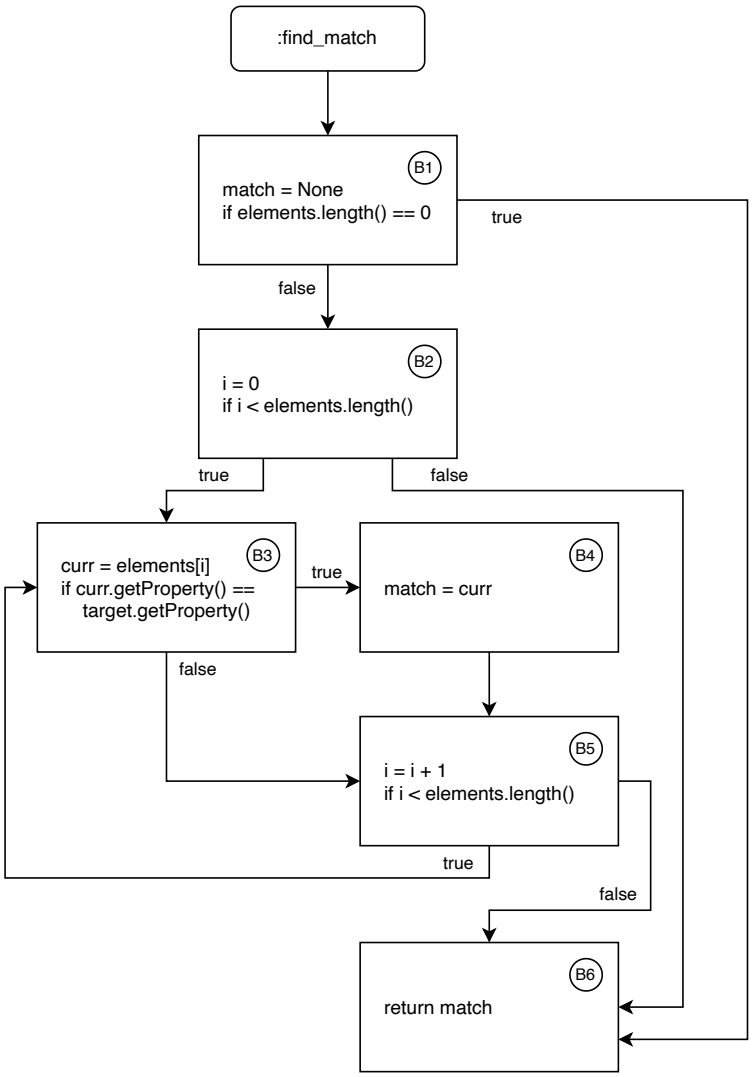
- Reference Counting
  - Für jedes Objekt wird eine Referenzzähler angelegt
  - Fällt der Zähler auf 0 wird das Objekt gelöscht / der Speicherplatz freigegeben
- Mark and Sweep
  - Co-routine mit dem Programm
  - Zwei Phasen: (1) alle erreichbaren Objekte markieren, (2) alle nichtmarkierten Objekte löschen
- Copying Collection
  - Heap in 2 oder mehr Teile aufteilen
  - Alle erreichbaren Objekte von einem Teil (old part / from-space) in den Anderen (new part / to-space) kopieren

(f) Zeichnen Sie *Basics Blocks* entsprechend dem folgenden Pseudocode. Zeichnen Sie dabei größtmöglich zusammenhänge Blöcke. Eingabeparameter einer Funktion müssen nicht extra deklariert werden. [5 Punkte]

```
function find_match(elements, target)

match = None

if elements.length() == 0 then
    return match
end if
```

```
i = 0

while i < elements.length() do
    curr = elements[i]

    if curr.getProperty() == target.getProperty() then
        match = curr
    end if

    i = i + 1
end while

return match
end function
```

Beispiellösung:



(g) Benennen und wenden Sie drei verschiede aus der Vorlesung bekannte Optimierungen exemplarisch auf den folgenden Pseudocode an. [3 Punkte]

```
int j = 0
int k = 5
int ret =  4
int runs = 5

while j < runs:
    k = 8
    ret = ret + (k / 2)
    if j > 2:
        ret = ret - (k / 2)
    j = j + 1
print(ret)
```

Beispiellösung:

Beispielsweise:

Constant Propagation

```
int ret =  ret + (8 / 2)
```

Constant Folding

```
int ret =  ret + 4
```

Strength Reduction

```
(k/2) $\rightarrow$  k >> 1
```

Loop unrolling

```
k = 8
ret = ret + (k / 2)
if j > 2:
    ret = ret - (k / 2)
j = j + 1

k = 8
ret = ret + (k / 2)
if j > 2:
    ret = ret - (k / 2)
j = j + 1

...
```

Dead Code Elimination

Code Motion

```
k = 8
while j < runs:
    ...
```

Wahrscheinlich nicht anwendbar: Copy Propagation, Common Subexpression Elimination, Algebraic Optimization, Inline Expansion Vollständig optimiert:

```
print(16)
```

(h) Wann ist ein Programm typsicher? [1 Punkt]

Beispiellösung:

Wenn seine Ausführung nicht zu Typfehlern führt. ODER Wenn es typkorrekt und in einer stark getypten Sprache geschrieben ist. (Typkorrektheit impliziert Typsicherheit in einer stark getypten Sprache.)